LABORATORIO N°3 B

"Simulador funcional con interfaz web: enlace de módulos con .EXTERNAL y punto fijo Q_n para la ISA LC-3"

FACULTAD DE INGENIERÍA - UNAM

CARRERA:

Ingeniería en Computación

MATERIA:

IC221 - Arquitectura de computadoras

DOCENTES:

Dr. Ing. Roberto E. Carballo

Dr. Ing. Aldo J. Benitez.

Ing. Roberto N. Schuster

Ing. Patricia V. Feltan

Ing. Gustavo O. Rossler

Ing. Eduardo Díaz

Sr. Joaquín Nahuel Giménez

Sr. Joaquín Zuk

ALUMNOS:

Leandro Leonel Grumicker

Rodrigo Martin Malczewski

Lautaro Nicolás Schneider

GRUPO: 8

ÍNDICE

TABLA DE CONTENIDOS

Tabla de Contenidos 2
1. PROBLEMA N°1: ENLACE DE MÓDULOS CON .EXTERNAL 3
1.1. Objetivo 3
1.2. Sintaxis y reglas de .EXTERNAL 3
1.3. Convenciones de llamada de módulos 3
1.4. Proceso de enlace 4
1.5. Ejemplo: ubicación en memoria 5
1.6. Llamadas entre módulos: .FILL + LD + JSRR 6
1.7. Errores y restricciones 6
2. PROBLEMA N°2: PUNTO FIJO Qn EN LC-3 SIMULATOR 7
2.1. DESARROLLO 7
2.2. Idea 7
2.3. Código comentado 7
2.4. Ejemplos 9
2.5. Detalle a tener en cuenta 9
3. CONCLUSIONES 10
4. REFERENCIAS BIBLIOGRÁFICAS 11

1. PROBLEMA N°1: ENLACE DE MÓDULOS CON .EXTERNAL

1.1. Objetivo

El objetivo de este trabajo es documentar cómo el simulador web enlaza módulos assembly de la LC-3 mediante la directiva *.EXTERNAL*, introducida en el capítulo 7 de *Introduction to Computing Systems (Patt & Patel, 2020),* de modo que el código que aporta cada grupo al *benchmark* pueda llamarse entre archivos.

Se han realizado algunas modificaciones a la especificación de esta directiva, principalmente con el fin de ayudar con la tarea de importar decenas de módulos en un solo programa, y evitar colisiones que puedan surgir entre nombres comúnes. Las reglas desarrolladas se especifican en la siguiente sección.

Una versión funcional del simulador está alojada en la URL https://lc3.lgrum.xyz/, con el código fuente disponible en un repositorio de GitHub. Además, incluye una guía de módulos con más detalle y ejemplos.

1.2. Sintaxis y reglas de .EXTERNAL

La declaración tiene la forma `.EXTERNAL MÓDULO-ETIQUETA`. Las reglas son:

- **Módulo y etiqueta.** El módulo se identifica por el nombre base o raíz del archivo (es decir, sin la extensión .asm):
  - TRIG-SIN se refiere a la etiqueta SIN del archivo **trig.asm**.
- **Mayúsculas.** El nombre del módulo no distingue mayúsculas de minúsculas; el de la etiqueta, sí.
- **Alcance de la directiva.** .EXTERNAL solo declara la dependencia: no llama a nada ni ocupa memoria. Debe declararse antes de usar la etiqueta. Es análogo a la palabra clave `import` presente en algunos lenguajes de programación.
- **Etiquetas locales.** No se importan: se usan directamente.

1.3. Convenciones de llamada de módulos

Para facilitar el uso de módulos externos, se establecieron una serie de convenciones a la hora de llamar a subrutinas externas. Estas se pueden encontrar a más detalle en la guía.

- El nombre del archivo ya es el nombre del módulo.
- `.ORIG` es opcional y se ignora en los módulos.
- R0, R1 y R2 se utilizan como argumentos.
- El resultado de la operación (si existe) se almacena en R0.
- La subrutina debe conservar el estado previo de R1 a R5 (callee-saved).
- R6 es el puntero de pila. Debe tener el mismo valor al salir que cuando entró.
- R7 es la dirección de retorno. Debe conservarse si se llama a otra subrutina.

1.4. Proceso de enlace

El proceso de enlace está basado en el proceso detallado en el capítulo 8 de *Patt & Patel*, resumido en Figura 1.1: Proceso de enlace de módulos y Figura 1.2: Recorrido de los módulos.

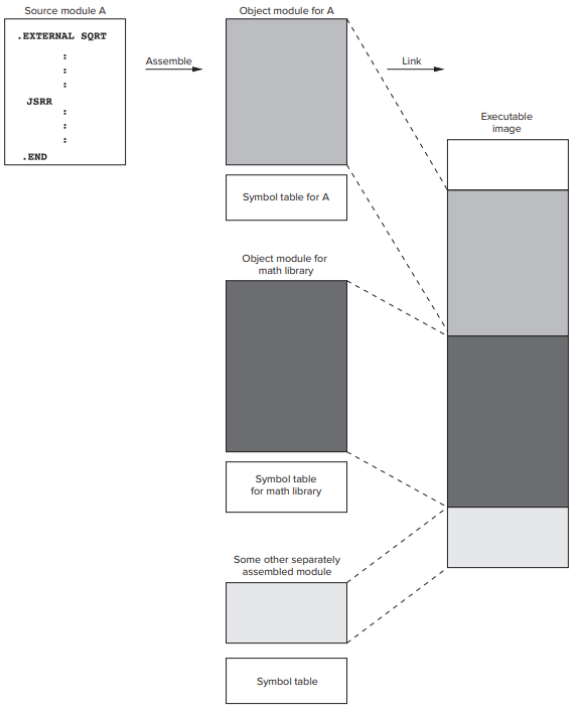


Figura 1.1: Proceso de enlace de módulos

Nota. Tomado de Introduction to Computing Systems: from bits to C/C++ & beyond, por Patt, Y. N., & Patel, S. J., 2020, McGraw-Hill

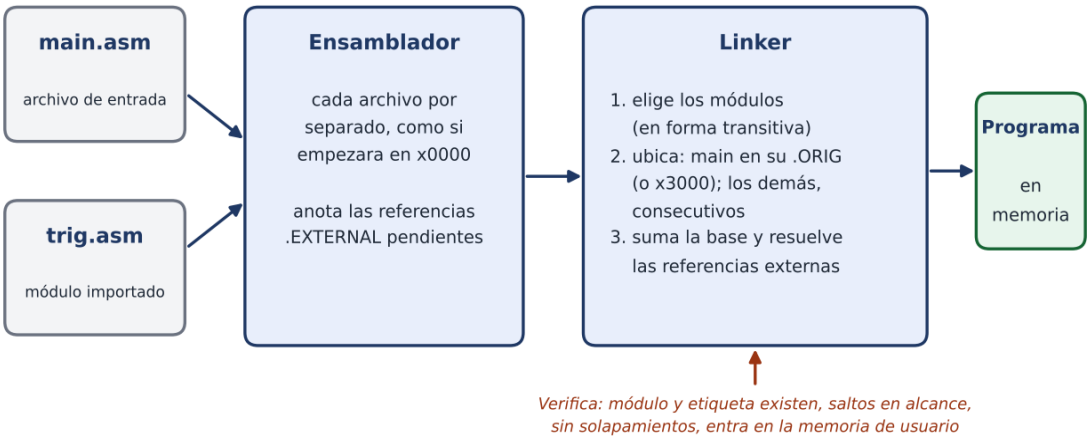


Figura 1.2: Recorrido de los módulos

1. **Ensamblado por módulo.** Cada archivo se ensambla por separado, como si empezara en la dirección cero. Se guardan las etiquetas locales y quedan anotadas las referencias externas pendientes: .EXTERNAL TRIG-SIN indica que SIN debe buscarse en el módulo **trig**.
2. **Selección de módulos.** El linker parte del archivo de entrada (main.asm), sigue sus .EXTERNAL y también las de esos módulos, de forma transitiva. El orden en que quedan depende del orden de las declaraciones. Un archivo que nadie importa no entra en el programa.
3. **Ubicación.** El programa de entrada se coloca en su .ORIG —o en x3000 si no tiene uno— y los demás módulos van consecutivos, sin espacios. En los módulos importados el .ORIG se ignora.
4. **Reubicación y resolución.** El linker suma la base asignada a cada módulo a las referencias absolutas locales y reemplaza cada referencia externa por la dirección final de la etiqueta importada: base de trig + desplazamiento de SIN dentro de trig. Así, .FILL TRIG-SIN se convierte en esa dirección completa de 16 bits. Los saltos relativos dentro de un módulo siguen funcionando aunque el linker lo mueva, porque la distancia entre la instrucción y su destino no cambia.

1.5. Ejemplo: ubicación en memoria

Para seguir la cuenta se escribieron dos módulos. main.asm es el archivo de entrada y llama a SIN, que está en trig.asm.

main.asm

```
        .ORIG x3000
        .EXTERNAL TRIG-SIN        ; declara que SIN está en el módulo trig
        LD    R1, P_SIN           ; R1 = dirección completa de SIN
        JSRR  R1                  ; llama a SIN (la vuelta queda en R7)
        HALT
P_SIN   .FILL TRIG-SIN            ; la dirección la pone el linker
        .END
```

trig.asm

```
        .ORIG x4000               ; se ignora al enlazar, emite una advertencia
SIN     ADD   R0, R0, #0          ; (cuerpo de la rutina)
        RET
        .END
```

Con estos archivos, **main.asm** ocupa cuatro palabras (LD, JSRR, HALT y .FILL), de x3000 a x3003. **trig** queda a continuación, así que su base es x3004. Como SIN está en el desplazamiento 0 de **trig**, su dirección final es x3004 + 0 = x3004, y esa es la palabra que reemplaza a .FILL TRIG-SIN. muestra el resultado.

Dirección	Palabra	Contenido	Nota
x3000	x2202	LD R1, P_SIN	PC = x3001; P_SIN está en +2
x3001	x4040	JSRR R1	salta a la dirección de R1
x3002	xF025	HALT	
x3003	x3004	P_SIN: .FILL TRIG-SIN	completada por el linker

Dirección	Palabra	Contenido	Nota
x3004	x1020	SIN: ADD R0, R0, #0	base de trig + 0
x3005	xC1C0	RET	

Tabla 1.1: Programa enlazado del ejemplo (codificación según la ISA de la LC-3 (Patt & Patel, 2020))

1.6. Llamadas entre módulos: .FILL + LD + JSRR

.FILL MÓDULO-ETIQUETA guarda la dirección completa de la etiqueta importada. En cambio, JSR, BR, LD, ST y las demás instrucciones con desplazamiento relativo solo pueden usarla si el destino está dentro de su alcance: JSR admite de −1024 a +1023 palabras, y BR, LD y ST de −256 a +255 [2]. Como el linker coloca los módulos uno detrás de otro, no hay garantía de que la rutina externa quede tan cerca.

Por eso, para llamar a una rutina de otro módulo resulta conveniente el patrón que muestra la guía:

- **.FILL** guarda la dirección completa de 16 bits (P_SIN en el ejemplo).
- **LD** trae esa dirección a un registro; P_SIN está en el mismo módulo, dentro del alcance de LD.
- **JSRR** salta a la dirección que hay en el registro, sin límite de distancia.

JSRR deja la dirección de retorno en R7. Si la rutina que llama es a su vez una subrutina, tiene que resguardar R7 antes de la llamada.

1.7. Errores y restricciones

Si algo no coincide, el enlace falla con un diagnóstico. Los casos que se detectan son:

- falta el módulo importado;
- la etiqueta no existe en ese módulo;
- se usa una etiqueta externa sin haberla declarado con .EXTERNAL;
- un salto relativo queda fuera de alcance;
- los módulos se solapan o no caben en la memoria de usuario.

Además, hay dos restricciones. Un módulo no puede importar desde el archivo de entrada ni desde el sistema operativo (a este se accede con TRAP). Y en la compilación administrativa, ciertas importaciones faltantes pueden convertirse en *stubs* (un marcador de posición, permite que el programa compile aún con etiquetas faltantes) de una palabra RET, de modo que la llamada simplemente regresa al código que la llamó, efectivamente ignorando la etiqueta faltante.

2. PROBLEMA N°2: PUNTO FIJO Qn EN LC-3 SIMULATOR

2.1. DESARROLLO

Se solicitó que los grupos con funciones en punto fijo expliciten el formato, el rango y la precisión. El simulador interpreta y muestra palabras en formato Q_n; esta sección documenta las funciones de conversión. Las secciones de código fueron extraídas directamente del código fuente del simulador, con comentarios agregados para mayor claridad. Estos fragmentos de código están escritos en TypeScript, un lenguaje de programación de Microsoft basado en JavaScript.

2.2. Idea

Una palabra de 16 bits en complemento a 2 se puede reinterpretar con n bits fraccionarios: es el formato Q_n. El bit 15 sigue siendo el signo y los últimos n bits están después de la coma binaria. El valor real es:

```
valor = entero_con_signo / 2^n
```

- **Q0** es un entero común con signo.
- **Q15** cubre el rango [-1, 1) con paso 1/32768.
- **Rango y paso:** de −2^(15−n) a 2^(15−n) − 2^−n, en pasos de 2^−n. Ej.: Q_8 cubre de −128 a 127.996 con paso 1/256.

  Ejemplo en Q8: la palabra `0x00C0` vale 192, y 192 / 2^8 = **0.75**.

2.3. Código comentado

Las funciones llevan un comentario en cada paso; las líneas que empiezan con // son las explicaciones.

Funciones de apoyo

```
// Pasa la palabra a entero con signo (complemento a 2).
export function signed(value: number): number {
  // Se queda solo con los 16 bits bajos.
  const word = value & 0xffff;
  // Bit 15 en 1 = negativo: se resta 65536 (0x10000).
  // Ej.: 0xFFFF -> 65535 - 65536 = -1.
  return word & 0x8000 ? word - 0x10000 : word;
}

// Mayor cantidad de bits fraccionarios: de Q0 a Q15.
export const MAX_FRACTION_BITS = 15;

// Cantidad máxima de decimales que se muestran en pantalla.
// Con 6 se distinguen todos los valores de Q15.
const FIXED_DECIMALS = 6;
```

`fixedPoint(value, n)` devuelve `signed(value) / 2**n` (0x00C0 en Q8 → 192 / 2^8 = 0.75). `trimZeros(text)` saca los ceros finales de un decimal ("0.750" → "0.75", "2.000" → "2") y no toca enteros como "100".

formatFixed: palabra → texto

```
export function formatFixed(
  value: number,         // la palabra de 16 bits
  fractionBits: number,  // la n de Qn
): { text: string; exact: string } {
  // Palabra -> número real. Ej.: 0xFF40 en Q8 -> -0.75.
  const number = fixedPoint(value, fractionBits);

  // toFixed(n) es exacto: un múltiplo de 1/2^n tiene como
  // máximo n decimales y un double lo representa sin error.
  const exact = trimZeros(number.toFixed(fractionBits));

  // Lo mismo, pero con a lo sumo 6 decimales, para que
  // Q15 no dé textos larguísimos.
  const rounded = trimZeros(
    number.toFixed(Math.min(fractionBits, FIXED_DECIMALS)),
  );

  // Si el redondeo no perdió nada, se muestra tal cual.
  // Si perdió algo, se antepone ≈. Siempre se devuelve
  // también exact, con el valor completo.
  return { text: rounded === exact ? exact : `≈${rounded}`, exact };
}
```

parseFixed: texto → palabra

```
export function parseFixed(
  text: string,          // lo que escribió el usuario
  fractionBits: number,  // la n de Qn
): number | undefined {
  // Saca espacios y un # inicial (notación LC-3: #0.25).
  const input = text.trim().replace(/^#/, "");

  // Q0 no usa decimales. La regex exige un punto decimal:
  // acepta 0.75, -1.5, 3. y .5; rechaza 42 (eso lo
  // interpreta el parser de enteros, parseWord).
  if (fractionBits === 0 || !/^-?(\d+\.\d*|\.\d+)$/.test(input))
    return undefined;

  // Pasa a escala entera y redondea al Qn más cercano.
  // Ej.: en Q8, 0.75 * 256 = 192.
  const raw = Math.round(Number(input) * 2 ** fractionBits);

  // Debe entrar en 16 bits con signo: -32768 a 32767.
  if (raw < -0x8000 || raw > 0x7fff) return undefined;

  // La máscara de 16 bits da el complemento a 2 en negativos.
  // Ej.: -192 & 0xffff = 0xFF40.
  return raw & 0xffff;
}
```

2.4. Ejemplos

Ida y vuelta en Q8:

```
parseFixed("-0.75", 8)
  -0.75 * 256 = -192
  -192 & 0xffff = 0xFF40

formatFixed(0xFF40, 8)
  signed(0xFF40) = -192
  -192 / 256 = -0.75
  → { text: "-0.75", exact: "-0.75" }
```

Tabla 2.1 reúne otras conversiones en distintos formatos.

Tabla 2.1: Palabras y valores en distintos formatos Qn.

Entrada	Formato	Palabra	Valor
`0.75`	Q8	`x00C0`	0.75
`-0.75`	Q8	`xFF40`	-0.75
`0.5`	Q15	`x4000`	0.5
`-1.`	Q15	`x8000`	-1
`0.25`	Q2	`x0001`	0.25

2.5. Detalle a tener en cuenta

Math.round redondea los empates .5 hacia +∞ (−2.5 → −2), no hacia el par más lejos de cero. Importa si se comparan resultados con otra implementación: round() de Python redondea al par y round() de C lejos de cero, así que ante un empate exacto pueden dar una palabra distinta.

3. CONCLUSIONES

El enlace entre módulos permite que cada grupo entregue su rutina en un archivo propio y que el benchmark se arme a partir de las dependencias declaradas con .EXTERNAL. El mecanismo se resume en tres ideas: cada módulo se ensambla como si empezara en la dirección cero, el linker lo ubica a continuación del anterior, y las referencias externas se reemplazan por la dirección final de la etiqueta importada.

Como las instrucciones relativas tienen un alcance limitado, la llamada a una rutina de otro módulo se resuelve con .FILL + LD + JSRR, que no depende de la distancia entre módulos. Además, el linker informa los módulos o etiquetas faltantes, los saltos fuera de alcance y los módulos que se solapan, de modo que esos errores aparecen al enlazar.

En punto fijo Qn, una palabra de 16 bits se interpreta como un entero con signo dividido por 2^n. Con ese criterio, el simulador convierte en ambos sentidos entre palabras y texto decimal, y permite verificar el rango y la precisión de cada formato.

4. REFERENCIAS BIBLIOGRÁFICAS

Patt, Y. N., & Patel, S. J. (2020). *Introduction to Computing Systems: from bits to C/C++ & beyond, Third Edition.* New York: McGraw-Hill.